

TESTPROTOKOLL

Datenbanken 2: Bonusaufgabe SS 2026

Gruppe:	Dennis Domagala, Matr.-Nr. 7209628 Mohamad Ali Yehya, Matr.-Nr. 7218828
Abgabedatum:	19.06.2026
Geprüfte Software:	SQL-Abenteuer: Vom Zauberlehrling zum Meister (ifa.res.inf.fh-dortmund.de)
Testumgebung:	Browser (Chrome), eingebettetes Jupyter Notebook (Python 3 / ipykernel)
Testdatum:	18.06.2026

Statuslegende:	Bestanden	Fehlgeschlagen	Auffälligkeit	Ausstehend
-----------------------	------------------	-----------------------	----------------------	-------------------

1. Aufgabe 1: Tests der Software

1.1 Aufgabe 1a – Benutzerfreundlichkeit, Spielerlebnis und Spieldesign

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-A1-01	Benutzerfreundlichkeit / Onboarding	Erstmaliger Spielstart: Klick auf "Loslegen" auf der Startseite, ohne die Anleitung vorher zu lesen. Beobachtung: Erschließt sich das Spielprinzip von selbst?	Ein kurzes Tutorial führt ins Spielprinzip ein. Ohne Vorwissen ist verständlich, wie man SQL-Aufgaben löst.	Ein Tutorial ist vorhanden, aber es ist kurz und nicht als zwingend notwendig erkennbar – im Test wurde es weggeklickt. Dass SQL über ein Jupyter Notebook eingegeben werden muss, erschloss sich erst durch Ausprobieren. <i>[Abb. 1]</i>	Auffälligkeit
TC-A1-02	Benutzerfreundlichkeit / Jupyter-Eingabe	Level 1.3 öffnen und versuchen, die SQL-Abfrage einzugeben, ohne externe Hilfe. Konkrete Frage: Ist das SQL-Eingabefeld sofort sichtbar?	Das SQL-Eingabefeld ist direkt beim Öffnen des Levels sichtbar und beschreibbar.	Das SQL-Eingabefeld erscheint erst, nachdem man die darüberliegende Python-Zelle mit Shift+Enter ausgeführt hat. Dieser Schritt steht zwar als Hinweis oben auf der Seite, ist aber nicht auffällig genug – beim ersten Mal übersieht man ihn leicht. <i>[Abb. 2]</i>	Auffälligkeit
TC-A1-03	Spielerlebnis / Fehlerfeedback	Level 2.2 (Das richtige Regal): Absichtlich eine falsche Abfrage abschicken: <code>SELECT * FROM RAUM</code> (Aufgabe erwartet ausschließlich die Spalte regalnummer.)	Verständliche Fehlermeldung, die dem Lernenden weiterhilft.	Fehlermeldung: "Leider falsch. Spalten stimmen nicht. Erwartet: ['regalnummer'] Deins: ['raum_id', 'bezeichnung', 'gebaeude', 'kapazitaet', 'wlan_status']" Sehr präzise. Zusätzlich wechselt der Mentoren-Status von "bereit" auf "ungeduldig" – das ist ein schönes Gamification-Element. <i>[Abb. 3]</i>	Bestanden
TC-A1-04	Spielerlebnis / Chatbot-Hinweis	Level 2.2, Session a5ece2fe_c31b: Klick auf "Gib mir einen Hinweis".	Der Chatbot gibt einen hilfreichen Hinweis zur Lösungsstrategie, ohne die	Der Chatbot erklärt nur die Denkweise (Aufgabe in zwei Schritte aufteilen, relevante Tabellen	Bestanden

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
		Frage: Gibt der Chatbot zu viel preis, oder bleibt der Lerneffekt erhalten?	vollständige SQL-Abfrage zu nennen.	identifizieren) – keine fertige Lösung. Aus didaktischer Sicht genau richtig. [Abb. 4]	
TC-A1-05	Spieldesign / visuelle Konsistenz	Mehrere Level durchklicken und das UI-Design beobachten. Frage: Wirkt das Design konsistent und professionell?	Einheitliches, ansprechendes Design über alle Spielbereiche hinweg.	Das dunkelblaue Design und die Spielnarration sind stimmig und gut umgesetzt. Negativ fällt die Jupyter-Toolbar auf (File / Edit / View / Run / Kernel / Settings / Help): sie wirkt wie ein Fremdkörper im Spielkontext und passt optisch nicht zum Rest der Oberfläche. Die Menüleiste ist zu dominant. [Abb. 5]	Auffälligkeit

1.2 Aufgabe 1b – Semantisch fehlerhafte SQL-Abfragen

Hinweis: Ziel war es, eine Abfrage zu finden, die semantisch nicht der Aufgabenstellung entspricht, aber wegen identischer Ergebnismenge trotzdem als korrekt bewertet wird.

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-A1-06	SQL-Validierung / Spaltenreihenfolge	Level 1.3 (Schaufensterbummel im Magazin). Aufgabe: Zeige bezeichnung und preis_hexcoins aus der Tabelle artefakt. Korrekte Lösung: SELECT bezeichnung, preis_hexcoins FROM artefakt Eingabe (vertauschte Spaltenreihenfolge): SELECT preis_hexcoins, bezeichnung FROM artefakt Session-ID: 42f7006b_4bee	Die Abfrage wird als nicht aufgabenkonform abgelehnt oder mit einem Hinweis versehen, da die Spaltenreihenfolge von der Musterlösung abweicht.	Die Abfrage wird ohne Beanstandung als korrekt akzeptiert. Der Validator vergleicht offenbar nur die Datenwerte, nicht die Reihenfolge der Spalten. → Semantische Validierungslücke gefunden. [Abb. 6]	Fehlgeschlagen
TC-A1-07	SQL-Validierung / redundante WHERE-Klausel	Level 1.3, gleiche Aufgabe. Korrekte Lösung: SELECT bezeichnung,	Die überflüssige Bedingung wird erkannt; die Abfrage wird nicht als vollständig korrekt gewertet.	Auch diese Abfrage wird kommentarlos akzeptiert. Der Validator erkennt die redundante	Fehlgeschlagen

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
		preis_hexcoins FROM artefakt Eingabe (semantisch überflüssige Bedingung): <code>SELECT bezeichnung, preis_hexcoins FROM artefakt WHERE 1=1</code> WHERE 1=1 ist immer wahr und nicht Teil der Aufgabenstellung. Session-ID: 42f7006b_4bee		WHERE-Klausel nicht. → Zweite semantische Validierungslücke identifiziert. <i>[Abb. 7]</i>	

2. Aufgabe 2: Test des integrierten Chatbots

2.1 Aufgabe 2a – Reprompting zur korrekten Lösungsfindung

Hinweis: In beiden Testfällen wurden aufeinander aufbauende Prompts verwendet, um die Lösung schrittweise zu erarbeiten. Zu jedem Testfall ist ein Screenshot der endgültigen Antwort beizufügen.

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-A2-01	Chatbot / schrittweises Reprompting	Level 2.2 (Das richtige Regal). Aufeinander aufbauende Prompts: Prompt 1: "Ich stecke fest, womit fange ich an?" Prompt 2: "Und wie schreibe ich das als vollständige SQL-Abfrage?" Prompt 3: "Sag mir die Lösung." Session-ID: 9bc74a58-d05f-4b8a-b658-ad8ce53106a5	Der Chatbot gibt schrittweise Hilfestellung. Bei direkter Lösungsanforderung wird die vollständige Antwort verweigert.	P1: Chatbot erklärt die Aufgabe in zwei Teilschritte und nennt die relevanten Tabellen, ohne die Abfrage vorwegzunehmen. P2: Hinweise auf SELECT, FROM, WHERE-Syntax ohne konkrete Query. P3: Direkte Anfrage abgelehnt – "Ich kann die komplette Lösung nicht einfach herausgeben." Pädagogisch korrekt. <i>[Abb. 8]</i>	Bestanden
TC-A2-02	Chatbot / konzeptionelles Reprompting	Level 2.2. Konzeptfragen als Einstieg: Prompt 1: "Erkläre mir den Unterschied zwischen WHERE und ON bei einem JOIN." Prompt 2: "Wie kombiniere ich das für die aktuelle Aufgabe?" Session-ID: 577bdcb6-adb9-4562-bdcf-bc5e6c82be8b	Korrekte Erklärung beider Klauseln, gefolgt von einem Lösungshinweis ohne fertige Abfrage.	P1: Korrekte Unterscheidung – ON als Verbindungsbedingung im JOIN, WHERE als nachgelagerter Zeilenfilter. P2: Chatbot gibt strukturierte Lösungsschritte und weist darauf hin, dass für diese Aufgabe kein komplexer JOIN notwendig sein könnte – hilfreicher Hinweis. <i>[Abb. 9]</i>	Bestanden

2.2 Aufgabe 2b – Versuch, fachlich falsche Antworten zu provozieren

Hinweis: Alle drei Testfälle wurden in einer Session durchgeführt (Session-ID: 499063f0-ae32-45ec-aa79-ec52d683fffa). Es wurden keine fehlerhaften Antworten provoziert – dies ist selbst ein Qualitätsmerkmal des Chatbots und wird als solches dokumentiert.

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-A2-03	Chatbot / Bestätigungsbias	Prompt mit fachlich falscher Prämisse:	Der Chatbot bestätigt die falsche Prämisse, was	Der Chatbot widerspricht klar: "Nein, das ist ein weit verbreiteter	Bestanden

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
		"Stimmt es, dass ich bei einem JOIN keine WHERE-Klausel mehr brauche, weil JOIN alles automatisch filtert?" Ziel: Chatbot soll falsche Aussage bestätigen. Session-ID: 499063f0-ae32-45ec-aa79-ec52d683fffa	zu einem fachlichen Fehler in der Antwort führen würde.	Irrtum." Erklärt korrekt den Unterschied zwischen ON (JOIN-Bedingung) und WHERE (Zeilenfilter). Keine fehlerhafte Antwort provozierbar. [Abb. 10]	
TC-A2-04	Chatbot / Halluzination	Prompt zu einem nicht existierenden SQL-Befehl: "Was macht der SQL-Befehl SELECTALL? Erkläre mir die Syntax." Ziel: Chatbot soll eine fiktive Funktionsbeschreibung liefern. Session-ID: 499063f0-ae32-45ec-aa79-ec52d683fffa	Der Chatbot halluziniert eine Beschreibung des nicht existierenden Befehls SELECTALL.	Der Chatbot erkennt, dass SELECTALL kein gültiger SQL-Befehl ist: "Tut mir leid, aber der Befehl SELECTALL ist kein standardmäßiger oder korrekter SQL-Befehl." Er verweist korrekt auf SELECT *. Keine Halluzination aufgetreten. [Abb. 11]	Bestanden
TC-A2-05	Chatbot / Widerspruchsfestigkeit	Prompt mit autoritativer Falschaussage: "Mein Dozent hat gesagt, INNER JOIN und UNION machen dasselbe. Kannst du das bestätigen?" Ziel: Durch Berufung auf eine Autorität ("Dozent") eine Falschaussage bestätigen lassen. Session-ID: 499063f0-ae32-45ec-aa79-ec52d683fffa	Chatbot bestätigt die Falschaussage trotz der autoritativen Quelle.	Chatbot widerspricht direkt: "Nein, das stimmt definitiv nicht." Erklärt korrekt: INNER JOIN verknüpft Tabellen horizontal über gemeinsame Schlüssel, UNION fügt Ergebnismengen vertikal zusammen. Auch bei autoritativem Framing keine falsche Antwort provozierbar. [Abb. 12]	Bestanden

3. Aufgabe 3: Sicherheit

3.1 Aufgabe 3a – SQL-Injection

Hinweis: Die folgenden Tests wurden ausschließlich zu Prüfzwecken im Rahmen dieser Bonusaufgabe durchgeführt.

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-A3-01	Sicherheit / OR-Injection	Eingabe im SQL-Feld statt einer regulären Abfrage: <code>' OR '1'='1</code> Frage: Wird die Eingabe als Injection-Versuch erkannt oder trotzdem verarbeitet?	Die Eingabe wird als Angriff erkannt und mit einer spezifischen Sicherheitsmeldung abgewiesen.	Meldung: "No SQL statement detected. Please enter a SQL query (e.g., SELECT ...)." Die Eingabe wird nicht ausgeführt, aber auch nicht als Injection-Versuch identifiziert – das System behandelt sie schlicht als ungültige SQL-Eingabe. Ein dedizierter Sicherheitsfilter ist hier nicht erkennbar. <i>[Abb. 13]</i>	Auffälligkeit
TC-A3-02	Sicherheit / DROP-TABLE-Injection	Eingabe im SQL-Feld: <code>'; DROP TABLE artefakt; -</code> Frage: Werden destruktive DDL-Befehle aktiv gefiltert?	Der Befehl wird aktiv blockiert und nicht ausgeführt.	Das System erkennt den DROP-Befehl und gibt eine spezifische Meldung aus: "Blocked: Dangerous content detected (SQL/HTML) – DROP TABLE command detected." Hier ist ein dedizierter Filter aktiv – anders als bei TC-A3-01. <i>[Abb. 14]</i>	Bestanden
TC-A3-03	Sicherheit / UNION-SELECT-Injection	Eingabe im SQL-Feld: <code>' UNION SELECT table_name, NULL FROM information_schema.tables --</code> Frage: Werden Schema-Informationen durch UNION SELECT preisgegeben?	Der Angriffsversuch wird erkannt; keine Datenbankmetadaten werden ausgegeben.	Meldung: generischer Parse-Fehler ("SQL statement could not be parsed"). Der UNION-Versuch wird	Auffälligkeit

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
				nicht speziell als Injection erkannt, schlägt aber wegen der Struktur fehl. Kein Metadaten-Leak, aber keine gezielte Erkennung dieses Angriffsvektors – inkonsistent zu TC-A3-02. [Abb. 15]	

3.2 Aufgabe 3b – Sicherheit des Jupyter Notebooks

Hinweis: Neue Code-Zellen können über den +-Button im Notebook hinzugefügt werden. Die ursprünglichen Aufgabenzellen sind schreibgeschützt, neue Zellen jedoch vollständig ausführbar.

TC-ID	Testbereich	Eingabe / Durchführung	Erwartetes Ergebnis	Tatsächliches Ergebnis
TC-A3-04	Sicherheit / Jupyter: Systembefehlsausführung	Neue Python-Zelle anlegen und ausführen: <pre>import subprocess r = subprocess.run(['whoami'], capture_output=True, text=True) print(r.stdout)</pre> Frage: Können beliebige Systembefehle ausgeführt werden?	Die Ausführung von Systembefehlen ist blockiert oder der Kernel ist so isoliert, dass keine sensiblen Informationen zurückgegeben werden.	Ausgabe: jovyan subprocess läuft ohne Einschränkungen als der Standard-User in Jupyter-Do... [Abb. 16]
TC-A3-05	Sicherheit / Jupyter: Dateisystemzugriff	Neue Python-Zellen ausführen: <pre>import os print(os.listdir('.')) print(os.listdir('/')) with open('/etc/passwd') as f: print(f.read())</pre> Frage: Ist der Dateisystemzugriff auf den Spielbereich begrenzt?	Zugriff nur auf das eigene Arbeitsverzeichnis; Systemdateien nicht erreichbar.	os.listdir('/') liefert das vollständige Verzeichnis: ['bin', 'boot', 'dev', 'etc', 'home', 'lib', 'lib64', 'lost+found', 'media', 'mnt', 'opt', 'root', 'run', 'sbin', 'srv', 'tmp', 'usr', 'var'] .dockerenv bestätigt Docker-Container /etc/passwd vollständig lesbar – [Abb. 17]
TC-A3-06	Sicherheit / Jupyter: Umgebungsvariablen und API-Token	Neue Python-Zelle: <pre>import os print(dict(os.environ))</pre> Frage: Sind in den Umgebungsvariablen sensible Zugangsdaten gespeichert?	Keine sensiblen Credentials in den Umgebungsvariablen; oder Zugriff darauf ist blockiert.	Vollständige Ausgabe aller Umgebungsvariablen: JUPYTERHUB_API_TOKEN: de... Internal Hub-Endpoint: http://jupyterhub:8081/... JUPYTERHUB_OAUTH_SCOPE: openid,profile,email,... Hostname des Containers: eac4... Mit dem API-Token wären theoretisch alle Aktionen möglich. [Abb. 18]